

Pseudocódigo del generador de horarios

Colegio Madres Dominicas (MMDD), Concepción

Taller de Gestión de Operaciones — Grupo 5

7 de octubre de 2026

Este documento describe en pseudocódigo el motor de asignación implementado en el paquete `notebooks/horarios/`: la clase `MotorHorarios` (`motor.py`, método `generar`) encadena el modelo de sesiones (`modelo.py`), la solución inicial (`construccion.py`) y la metaheurística de mejora (`metaheurísticas/min_conflicts_tabu.py`). El método construye el horario semanal de los 24 cursos regulares y la Educación Física de los 3 niveles de párvulos en tres fases: (i) *prepara* el modelo: fija las franjas de electivos de 3° y 4° medio, que comparten las secciones A y B, y corta la malla de cada curso en sesiones dobles de 90 minutos y simples de 45; (ii) ubica cada sesión con una heurística *constructiva voraz*, de la más restringida a la menos restringida, en la posición de menor costo; y (iii) repara las violaciones restantes con una búsqueda local *min-conflicts* con lista tabú, ruido aleatorio y reinicios desde la mejor solución, que intercambia sesiones dentro de un mismo curso.

Todas las restricciones se evalúan con una única función de costo, separada en *claves*: cada clave agrupa las sesiones que compiten por un mismo recurso (un docente en un bloque, una asignatura en un día, los recintos deportivos o una sala en un bloque), y su costo se actualiza de forma incremental cada vez que una sesión entra o sale.

Notación

- $\mathcal{C}$: los 27 cursos: 24 regulares (1° básico a 4° medio, secciones A y B) y 3 de párvulos (Prekínder A, Kínder A y B), de los que solo se programa la Educación Física.
- $\mathcal{D}$: días de lunes a viernes. Un bloque es un par $s = (d, b)$ con día d y número $b \in \{1, \dots, 10\}$; $B_{c,d}$: bloques de la jornada del curso c el día d .
- $\mathcal{P} = \{(1, 2), (3, 4), (5, 6), (7, 8), (9, 10)\}$: pares pedagógicos. Los recreos caen después de los bloques 2, 4, 6 y 8, así que un par nunca cruza un recreo.
- M_c : malla curricular del curso c , formada por tuplas (a, h, P, e) : asignatura, horas semanales, conjunto de docentes (más de uno si hay co-docencia) y espacio.
- Sesión $\sigma = (c_\sigma, a_\sigma, P_\sigma, e_\sigma, \lambda_\sigma)$: curso, asignatura, docentes, espacio y largo $\lambda_\sigma \in \{1, 2\}$ en bloques. Las sesiones de electivos son *fijas*; el resto son *movibles*. κ_σ : a quién atienden los docentes de σ (el curso, o el nivel completo en un electivo compartido por A y B); $\varphi(a)$: familia de la asignatura a (English skills cuenta como Inglés).
- $\text{pos}(\sigma)$: tupla de bloques que ocupa la sesión σ . V_c : celdas movibles del curso c (su jornada sin los bloques fijos); $Y_c(s)$: sesión que ocupa la celda $s \in V_c$, o nulo; Π_c : pares pedagógicos con sus dos celdas en V_c .
- Claves de restricción y sesiones registradas en cada una, $\text{occ}(k)$:
 - $D(p, s)$: sesiones que usan al docente p en el bloque s ;
 - $F(c, d, f)$: sesiones movibles del curso c de la familia f el día d ;
 - $G(s)$: sesiones de Educación Física en el bloque s ;
 - $R(r, s)$: sesiones que usan la sala de capacidad limitada r en el bloque s .
- z : costo total; v : número de violaciones duras; $\mathcal{M}$: claves con alguna violación dura; $\mathcal{B}$: claves con costo blando.
- Recintos deportivos: Gimnasio B para párvulos y 1° a 4° básico ($n_B(s)$ sesiones), Gimnasio A para 5° básico a 4° medio ($n_A(s)$ sesiones). Cada gimnasio recibe un curso por bloque y el Patio Santo Domingo recibe un excedente más, con costo blando.

- $[P]$: vale 1 si la proposición P es verdadera y 0 en caso contrario.
- Valores usados por el programa: semilla $\omega = 3$ (valor por defecto de main); $K = 150\,000$ pasos; peso duro $W = 1000$; peso del patio $w_P = 1$; capacidad de la Sala English skills $cap = 1$; probabilidad de movimiento aleatorio $\varepsilon = 0,03$; permanencia tabú $\tau \in \{10, \dots, 20\}$; reinicio cada $R = 2500$ pasos sin mejora; parada anticipada tras $U = 3000$ pasos sin mejora si el horario ya es factible.

Algoritmo 1 GENERARHORARIO($\mathcal{C}, \mathcal{D}, B, M, \omega, K$)

Entrada: $\mathcal{C}$: cursos; $\mathcal{D}$: días hábiles; $B_{c,d}$: bloques de la jornada del curso c el día d ; M_c : malla curricular del curso c ; ω : semilla aleatoria; K : número máximo de pasos de la búsqueda local.

Salida: Horario H que asigna a cada curso y bloque una lección (a, P, e) , cubre exactamente las horas de M_c y minimiza el costo z ; es factible si $v = 0$.

- 1: inicializar el generador aleatorio con la semilla ω
 - 2: PREPARAR // Algoritmo 2
 - 3: CONSTRUCCIÓNVORAZ // Algoritmo 3
 - 4: MINCONFLICTOSTABÚ(K) // Algoritmo 4
 - 5: $H \leftarrow$ horario construido a partir de $\text{pos}(\sigma)$ para toda sesión σ
 - 6: **return** H
-

Algoritmo 2 PREPARAR()

Entrada: Cursos $\mathcal{C}$, jornadas B , mallas M y franjas de electivos de 3° y 4° medio.

Salida: Sesiones fijas registradas, celdas V_c , pares Π_c y lista de sesiones movibles.

- 1: $\Sigma \leftarrow \emptyset$; $z \leftarrow 0$; $v \leftarrow 0$; $\text{occ}(\cdot) \leftarrow \emptyset$; $\mathcal{M} \leftarrow \emptyset$; $\mathcal{B} \leftarrow \emptyset$; $L_c \leftarrow \emptyset$, $\forall c \in \mathcal{C}$
 - 2: **for all** nivel $n \in \{3^\circ \text{ medio}, 4^\circ \text{ medio}\}$, franja $j \in \{1, 2, 3\}$ de n **do** // grilla oficial 2026
 - 3: $P \leftarrow$ docentes de los grupos de la franja j // A y B mezclados en grupos
 - 4: **for all** periodo doble (d, b) de la franja j **do** // 3 periodos de 2 bloques
 - 5: crear la sesión fija σ con $P_\sigma = P$, $\lambda_\sigma = 2$ y $\kappa_\sigma =$ electivos de n
 - 6: $\text{pos}(\sigma) \leftarrow ((d, b), (d, b + 1))$; $\Sigma \leftarrow \Sigma \cup \{\sigma\}$
 - 7: $L_{nA} \leftarrow L_{nA} \cup \{(d, b), (d, b + 1)\}$; $L_{nB} \leftarrow L_{nB} \cup \{(d, b), (d, b + 1)\}$
 - 8: **end for**
 - 9: **end for**
 - 10: **for all** $c \in \mathcal{C}$ **do**
 - 11: $V_c \leftarrow \{(d, b) : d \in \mathcal{D}, b \in B_{c,d}\} \setminus L_c$; $Y_c(s) \leftarrow \text{nulo}$, $\forall s \in V_c$
 - 12: $\Pi_c \leftarrow \{((d, b), (d, b')) : d \in \mathcal{D}, (b, b') \in \mathcal{P}, (d, b) \in V_c, (d, b') \in V_c\}$
 - 13: **end for**
 - 14: **for all** $c \in \mathcal{C}$, $(a, h, P, e) \in M_c$ **con** a que no es electivo **do**
 - 15: **if** c es de párvulos **then**
 - 16: agregar a Σ h sesiones movibles $(c, a, P, e, 1)$
 - 17: **else**
 - 18: agregar a Σ $\lfloor h/2 \rfloor$ sesiones $(c, a, P, e, 2)$ y $(h \bmod 2)$ sesiones $(c, a, P, e, 1)$
 - 19: **end if**
 - 20: **end for**
 - 21: **for all** sesión fija $\sigma \in \Sigma$ **do**
 - 22: REGISTRAR($\sigma, \text{pos}(\sigma), +1$) // Algoritmo 7
 - 23: **end for**
-

Algoritmo 3 CONSTRUCCIÓN VORAZ()

Entrada: Sesiones Σ ya creadas y sesiones fijas registradas.

Salida: Cada sesión movable ubicada: $\text{pos}(\sigma)$ definido y Y_c completo.

```
1:  $\eta(p) \leftarrow$  horas semanales de grilla del docente  $p$  // cada franja de electivos suma 6
2:  $\Lambda \leftarrow$  sesiones movibles ordenadas por: // de la más a la menos restringida
    $\lambda_\sigma$  decreciente,  $|P_\sigma|$  decreciente,  $\max_{p \in P_\sigma} \eta(p)$  decreciente, primero  $e_\sigma = \text{Gimnasio}$ , y desempate aleatorio
3: for all  $\sigma \in \Lambda$ , en orden do
4:    $c \leftarrow c_\sigma$ 
5:   if  $\lambda_\sigma = 2$  then
6:      $O \leftarrow \{(s, s') \in \Pi_c : Y_c(s) = Y_c(s') = \text{nulo}\}$  // pares libres
7:   else
8:      $O \leftarrow \{(s) : s \in V_c, Y_c(s) = \text{nulo}\}$  // celdas libres
9:   end if
10:  barajar  $O$  al azar;  $S^* \leftarrow \text{nulo}$ ;  $\Delta^* \leftarrow \infty$ 
11:  for all  $S \in O$ , en ese orden do
12:     $\Delta \leftarrow \text{REGISTRAR}(\sigma, S, +1)$ ; REGISTRAR( $\sigma, S, -1$ ) // costo de ubicar  $\sigma$  en  $S$ 
13:    if  $\Delta < \Delta^*$  then
14:       $(S^*, \Delta^*) \leftarrow (S, \Delta)$ 
15:      if  $\Delta = 0$  then
16:        break // primera posición sin costo
17:      end if
18:    end if
19:  end for
20:  if  $S^* = \text{nulo}$  then
21:    error: no hay espacio en la jornada de  $c$ 
22:  end if
23:   $Y_c(s) \leftarrow \sigma, \forall s \in S^*$ ;  $\text{pos}(\sigma) \leftarrow S^*$ ; REGISTRAR( $\sigma, S^*, +1$ )
24: end for
```

Algoritmo 4 MINCONFLICTOSTABÚ(K)

Entrada: K : pasos máximos; horario inicial completo con sus claves, z , v , $\mathcal{M}$ y $\mathcal{B}$.

Salida: Mejor horario encontrado; factible si termina con $v = 0$.

```
1:  $\Theta(\cdot) \leftarrow -1$  // lista tabú: paso hasta el que una sesión no vuelve a una posición
2:  $z^* \leftarrow z$ ;  $\text{pos}^* \leftarrow \text{pos}$ ;  $u \leftarrow 0$  // mejor solución y pasos sin mejora
3: for  $k \leftarrow 0$  to  $K - 1$  do
4:   if  $z = 0 \vee (v = 0 \wedge u > U)$  then
5:     break // óptimo, o factible y estancado
6:   end if
7:    $\mathcal{K} \leftarrow \mathcal{M}$  si  $\mathcal{M} \neq \emptyset$ , si no  $\mathcal{B}$  // primero las violaciones duras
8:   si  $\mathcal{K} = \emptyset$  entonces break
9:    $\kappa \leftarrow$  clave aleatoria de  $\mathcal{K}$ ;  $\Sigma' \leftarrow \{\sigma \in \text{occ}(\kappa) : \sigma \text{ movable}\}$ 
10:  si  $\Sigma' = \emptyset$  entonces break
11:   $\sigma \leftarrow$  sesión aleatoria de  $\Sigma'$ ;  $c \leftarrow c_\sigma$ ;  $N \leftarrow \text{VECINOS}(\sigma)$  // Algoritmo 5
12:  si  $N = \emptyset$  entonces continue // el paso cuenta igual
13:  if  $\text{rand}() < \varepsilon$  then
14:     $\mu \leftarrow$  intercambio aleatorio de  $N$  // ruido: ignora costo y tabú
15:  else
16:     $\mu \leftarrow$  nulo;  $\Delta^* \leftarrow \infty$ 
17:    for all  $\pi \in N$  do
18:       $\Delta \leftarrow \text{MOVER}(c, \pi)$ ;  $\text{MOVER}(c, \pi^{-1})$  // evalúa y deshace
19:      if  $\Theta(\sigma, \pi(\text{pos}(\sigma))) > k \wedge z + \Delta \geq z^*$  then
20:        continue // tabú, salvo que mejore la mejor solución
21:      end if
22:      if  $\Delta < \Delta^* \vee (\Delta = \Delta^* \wedge \text{rand}() < \frac{1}{2})$  then
23:         $(\mu, \Delta^*) \leftarrow (\pi, \Delta)$  // mejor vecino, empates al azar
24:      end if
25:    end for
26:    si  $\mu = \text{nulo}$  entonces continue // todos tabú
27:  end if
28:   $S_0 \leftarrow \text{pos}(\sigma)$ ;  $\text{MOVER}(c, \mu)$  // se aplica aunque empeore
29:   $\Theta(\sigma, S_0) \leftarrow k + \tau$ , con  $\tau$  entero aleatorio en  $\{10, \dots, 20\}$ 
30:  if  $z < z^*$  then
31:     $z^* \leftarrow z$ ;  $\text{pos}^* \leftarrow \text{pos}$ ;  $u \leftarrow 0$ 
32:  else
33:     $u \leftarrow u + 1$ 
34:    if  $u \bmod R = 0$  then // reinicio
35:       $\text{RESTAURAR}(\text{pos}^*)$ 
36:      for  $i \leftarrow 1$  to  $15$  do // perturbación
37:         $c' \leftarrow$  curso aleatorio;  $\sigma' \leftarrow$  sesión aleatoria de  $c'$ 
38:         $\pi' \leftarrow$  intercambio aleatorio de  $\text{VECINOS}(\sigma')$ ;  $\text{MOVER}(c', \pi')$ 
39:      end for
40:    end if
41:  end if
42: end for
43: if  $z > z^*$  then
44:    $\text{RESTAURAR}(\text{pos}^*)$ 
45: end if
```

Algoritmo 5 $\text{VECINOS}(\sigma)$

Entrada: σ : sesión movable del curso $c = c_\sigma$.

Salida: Conjunto N de intercambios π : permutaciones de celdas de V_c que mueven σ .

```
1:  $N \leftarrow \emptyset$ ;  $s_1 \leftarrow$  primera celda de  $\text{pos}(\sigma)$ 
2: if existe  $\pi_1 \in \Pi_c$  que contiene a  $s_1$  then // intercambio de pares
3:   for all  $\pi_2 \in \Pi_c \setminus \{\pi_1\}$  do
4:      $N \leftarrow N \cup \{\pi_1 \leftrightarrow \pi_2\}$  // 1.ª celda con 1.ª celda y 2.ª con 2.ª
5:   end for
6: end if
7: if  $\lambda_\sigma = 1$  then // intercambio simple
8:   for all  $s_2 \in V_c \setminus \{s_1\}$  con  $Y_c(s_2) = \text{nulo} \vee \lambda_{Y_c(s_2)} = 1$  do
9:      $N \leftarrow N \cup \{s_1 \leftrightarrow s_2\}$ 
10:  end for
11: end if
12: return  $N$  // un doble siempre viaja en un par completo
```

Algoritmo 6 Claves de restricción y costo de cada clave

Entrada: Sesión σ con bloques candidatos S , o clave k con sus sesiones $\text{occ}(k)$.

Salida: Claves que toca σ en S , y costo y violaciones duras de una clave.

```
1: function CLAVES( $\sigma, S$ ) //  $S$ : bloques de un mismo día  $d$ 
2:    $Q \leftarrow \{D(p, s) : s \in S, p \in P_\sigma\}$ 
3:   if  $e_\sigma = \text{Gimnasio}$  then
4:      $Q \leftarrow Q \cup \{G(s) : s \in S\}$ 
5:   end if
6:   if  $e_\sigma$  es una sala de capacidad limitada  $r$  then
7:      $Q \leftarrow Q \cup \{R(r, s) : s \in S\}$ 
8:   end if
9:   if  $\sigma$  es movable then
10:     $Q \leftarrow Q \cup \{F(c_\sigma, d, \varphi(a_\sigma))\}$  // una sesión por familia y día
11:   end if
12:   return  $Q$ 
13: end function

14: function COSTOCLAVE( $k$ ) // retorna (costo, violaciones duras)
15:    $O \leftarrow \text{occ}(k)$ 
16:   if  $k = D(p, s)$  then
17:      $n \leftarrow \text{máx}\{0, |\{\kappa_\sigma : \sigma \in O\}| - 1\}$  // un electivo de A y B cuenta una vez
18:   else if  $k = F(c, d, f)$  then
19:      $n \leftarrow \text{máx}\{0, |O| - 1\}$ 
20:   else if  $k = R(r, s)$  then
21:      $n \leftarrow \text{máx}\{0, |O| - \text{cap}(r)\}$ 
22:   else //  $k = G(s)$ 
23:      $x \leftarrow \text{máx}\{0, n_B(s) - 1\} + \text{máx}\{0, n_A(s) - 1\}$  // excedente de los gimnasios
24:      $n \leftarrow \text{máx}\{0, x - 1\}$  // el patio absorbe un excedente
25:     return  $(Wn + w_P \text{mín}\{x, 1\}, n)$ 
26:   end if
27:   return  $(Wn, n)$ 
28: end function
```

Algoritmo 7 Operaciones incrementales sobre el horario

Entrada: Estado actual: pos , Y , occ , z , v , $\mathcal{M}$ y $\mathcal{B}$.

Salida: Estado consistente tras cada ubicación, retiro o intercambio, y la variación de costo Δ .

```
1: function REGISTRAR( $\sigma, S, \pm 1$ )                                     // agrega o quita  $\sigma$  de las celdas  $S$ 
2:    $\Delta \leftarrow 0$ 
3:   for all  $k \in \text{CLAVES}(\sigma, S)$  do
4:      $(c_0, n_0) \leftarrow \text{COSTOCLAVE}(k)$ 
5:      $\text{occ}(k) \leftarrow \text{occ}(k) \cup \{\sigma\}$  o  $\text{occ}(k) \setminus \{\sigma\}$ , según el signo
6:      $(c_1, n_1) \leftarrow \text{COSTOCLAVE}(k)$ ;  $\Delta \leftarrow \Delta + c_1 - c_0$ ;  $v \leftarrow v + n_1 - n_0$ 
7:     actualizar la pertenencia de  $k$  a  $\mathcal{M}$  (si  $n_1 > 0$ ) y a  $\mathcal{B}$  (si tiene costo blando)
8:   end for
9:    $z \leftarrow z + \Delta$ 
10:  return  $\Delta$ 
11: end function

12: function MOVER( $c, \pi$ )                                           //  $\pi$ : permutación de celdas de  $V_c$ 
13:   $A \leftarrow \{Y_c(s) : s \in \text{dom}(\pi), Y_c(s) \neq \text{nulo}\}$ ;  $\Delta \leftarrow 0$ 
14:  for all  $\sigma \in A$  do
15:     $\Delta \leftarrow \Delta + \text{REGISTRAR}(\sigma, \text{pos}(\sigma), -1)$ 
16:  end for
17:   $Y_c(\pi(s)) \leftarrow Y_c(s)$  para toda  $s \in \text{dom}(\pi)$ , en simultáneo
18:  for all  $\sigma \in A$  do
19:     $\text{pos}(\sigma) \leftarrow \pi(\text{pos}(\sigma))$ , ordenado;  $\Delta \leftarrow \Delta + \text{REGISTRAR}(\sigma, \text{pos}(\sigma), +1)$ 
20:  end for
21:  return  $\Delta$ 
22: end function

23: procedure RESTAURAR( $\text{pos}'$ )
24:   $\text{occ}(\cdot) \leftarrow \emptyset$ ;  $z \leftarrow 0$ ;  $v \leftarrow 0$ ;  $\mathcal{M} \leftarrow \emptyset$ ;  $\mathcal{B} \leftarrow \emptyset$ ;  $\text{pos} \leftarrow \text{pos}'$ 
25:  reconstruir  $Y_c$  a partir de  $\text{pos}$  para todo  $c \in \mathcal{C}$ 
26:  for all  $\sigma \in \Sigma$  do
27:     $\text{REGISTRAR}(\sigma, \text{pos}(\sigma), +1)$ 
28:  end for
29: end procedure
```
